

INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus



APP DEVELOPMENT

ASSIGNMENT NO 2 (Theory)

SUBMITTED TO:

SIR UZAIR HASSAN

SUBMITTED BY:

MARIA SALEEM
242201075

Department of Computer Science,
KICSIT Kahuta.

Table of Contents

- 1. Introduction
- 2. Embedding ML/DL Models in Android Applications
- 3. TensorFlow Lite (TFLite)
- 4. Step-by-Step Process of Using TFLite in Android
- 5. Overview and Purpose of the Provided GitHub Repository
- 6. Analysis of the GitHub Repository
- 7. Proposed AI-Based Application – Number Guessing App
- 8. Technologies, Tools, Libraries, and AI Model Used
- 9. System Design and Workflow
- 10. Main Features and Functionality
- 11. Implementation Methodology
- 12. Testing, Results, and Discussion
- 13. Conclusion

1 Introduction

Artificial Intelligence (AI) is becoming an important part of modern mobile application development. It allows mobile applications to perform intelligent tasks such as image recognition, voice recognition, text classification, and prediction.

Machine Learning (ML) and Deep Learning (DL) models can be integrated into Android applications to make them more intelligent and interactive. TensorFlow Lite (TFLite) is a lightweight framework that allows trained machine learning models to run directly on mobile devices.

This project focuses on developing a Number Guessing App, an AI-based Android application that recognizes handwritten digits using a TensorFlow Lite model named `digit_model_TFLite`.

The user can draw a number on the application screen, and the application processes the drawing and predicts which digit has been drawn. The model performs the prediction directly on the Android device.

The project also involves studying an existing GitHub repository to understand how TensorFlow Lite models are integrated into Android applications.

1.1 Project Objectives

The main objectives of this project are:

1. To understand the integration of Machine Learning and Deep Learning models into Android applications.
2. To study the working and importance of TensorFlow Lite.
3. To explore an existing GitHub repository and understand its source code.
4. To understand how a TensorFlow Lite model is loaded and executed in Android Studio.
5. To develop an AI-based Number Guessing App using the `digit_model_TFLite` model.
6. To understand how handwritten input is processed and converted into model predictions.
7. To display the predicted digit in a user-friendly interface.
8. To learn the practical implementation of on-device machine learning.

2 Embedding ML/DL Models in Android Applications

2.1 Meaning of Embedding ML/DL Models

Embedding a Machine Learning or Deep Learning model in an Android application means adding a trained AI model to a mobile application so that it can perform intelligent tasks.

Instead of sending all input data to an external server, the application can use the integrated model to process data directly on the mobile device.

For example, in the Number Guessing App, the user draws a handwritten digit on the screen. The application sends this drawing to the integrated AI model. The model analyzes the drawing and predicts the most likely digit between 0 and 9.

This process allows Android applications to provide intelligent features without depending completely on an internet connection.

2.2 Importance in Real-World Mobile Applications

Embedding ML/DL models is important because it makes mobile applications smarter and more useful.

1. **Image Recognition**
   AI models can identify objects, animals, plants, and other items from images. For example, a mobile application can recognize different objects using its camera.

2. **Text Classification**
   Machine Learning models can analyze text and classify it into different categories. For example, an application can identify spam messages or categorize customer feedback.

3. **Prediction Systems**
   AI models can analyze input data and generate predictions. For example, applications can predict student performance, recognize patterns, or estimate future results.

4. **Handwritten Digit Recognition**
   AI models can recognize handwritten numbers from images or drawings. The Number Guessing App uses this concept to identify digits drawn by the user.

2.3 Benefits of On-Device ML Inference

On-device inference means running an AI model directly on a mobile device instead of sending data to a remote server.

Its major benefits include:

1. **Low Latency**
   The model processes input directly on the device, which reduces communication delays and provides faster results.

2. **Offline Functionality**
   The application can perform predictions without requiring a continuous internet connection, provided the model and required resources are available locally.

3. **Privacy**
   User data can remain on the mobile device instead of being sent to an external server. This can improve data privacy.

4. **Reduced Server Cost**
   On-device processing reduces the need for cloud computing resources and server-side processing.

5. **Faster Response**
   The application can provide predictions quickly because it does not need to upload input data and wait for a server response.

2.4 Challenges of On-Device ML Inference

Although on-device ML provides many benefits, it also has some challenges.

1. **Limited Memory**
   Mobile devices have limited RAM and storage. Large AI models can consume significant memory.

2. **Battery Consumption**
   Running complex machine learning operations can consume additional battery power.

3. **Limited Processing Power**
   Mobile devices generally have fewer computing resources than powerful computers and servers.

4. **Model Size**
   Large models increase application size and may require more storage space.

5. **Device Compatibility**
   Different Android devices have different hardware specifications and processing capabilities. Therefore, model performance may vary between devices.

6. **Model Accuracy**
   A model may produce incorrect predictions when input data is unclear, poorly prepared, or different from its training data.

3 TensorFlow Lite (TFLite)

3.1 Definition of TensorFlow Lite

TensorFlow Lite (TFLite) is a lightweight machine learning framework designed to run trained Machine Learning and Deep Learning models on mobile devices and embedded systems.

It is part of the TensorFlow ecosystem and allows developers to convert trained TensorFlow models into a smaller and more efficient format called `.tflite`.

These converted models can be integrated into Android applications to perform predictions directly on the device.

TensorFlow Lite is especially useful for applications that require fast predictions, offline functionality, and efficient resource usage.

In this project, TensorFlow Lite is used to run the `digit_model_TFLite` model inside the Number Guessing App.

3.2 Difference Between Standard TensorFlow and TensorFlow Lite

Standard TensorFlow	**TensorFlow Lite**
Mainly used for developing and training ML models.	Mainly used for deploying trained models on devices.
Designed for computers, servers, and cloud environments.	Designed for mobile and embedded devices.
Can require significant computing resources.	Optimized for limited device resources.
Models may have larger deployment requirements.	Uses a lightweight `.tflite` model format.
Supports model training and inference.	Primarily focuses on model inference.
Commonly used for complex ML development.	Commonly used for mobile AI applications.

3.3 Importance of TensorFlow Lite in Android Applications

TensorFlow Lite is suitable for Android applications for the following reasons:

1. **Better Performance**
   It is designed to perform machine learning operations efficiently on mobile devices.

2. **Smaller Model Size**
   TensorFlow Lite supports model optimization and quantization techniques that can reduce model size.

3. **Efficient Resource Usage**
   It is designed to work with limited memory and processing resources.

4. **Offline Predictions**
   It allows applications to perform inference without continuously communicating with online servers.

5. **Hardware Acceleration**
   It supports suitable hardware acceleration options, which can improve inference performance on supported devices.

6. **Easy Android Integration**
   Developers can add the TensorFlow Lite library to an Android Studio project and use its Interpreter to execute a model.

4 Step-by-Step Process of Using TensorFlow Lite in Android

Integrating a TensorFlow Lite model into an Android application involves several important steps.

4.1 Converting a TensorFlow Model into TFLite Format

The first step is to develop and train a machine learning model using TensorFlow.

After training, the model can be converted into TensorFlow Lite format using the TensorFlow Lite Converter.

The converted model is saved with the `.tflite` extension. It can then be added to an Android application.

The conversion process helps prepare the trained model for efficient execution on mobile devices.

4.2 Adding the Model and Dependencies in Android Studio

After conversion, the `.tflite` model file is added to the Android project's assets folder.

The TensorFlow Lite library is also added to the app-level Gradle configuration.

After adding the required dependency, Gradle Sync is performed to download and configure the library.

This allows the Android application to access and execute the TensorFlow Lite model.

4.3 Role of the TensorFlow Lite Interpreter

The TensorFlow Lite Interpreter is responsible for executing the trained model.

It loads the `.tflite` model and provides an interface between the Android application and the machine learning model.

Its main responsibilities include:

- Loading the trained model.
- Accepting prepared input data.
- Running model inference.
- Producing prediction results.
- Returning the output to the application.

The Interpreter allows the Android application to use the trained model without training it again.

4.4 Preparing Input Data

Before sending data to the model, the input must be prepared according to the model's requirements.

For handwritten digit recognition, the application performs image preprocessing.

Common preprocessing steps include:

- Capturing the user's drawing.
- Converting the drawing into an image or bitmap.
- Resizing the image according to the model's input dimensions.
- Converting the image into grayscale if required.
- Normalizing pixel values if required by the model.
- Converting the processed data into the correct input format.

A common handwritten digit recognition model uses a 28×28 grayscale image. However, the exact input dimensions and data type depend on the selected model.

4.5 Running Inference

Inference means using a trained machine learning model to make predictions on new input data.

After preparing the input, the Android application passes it to the TensorFlow Lite Interpreter.

The Interpreter processes the input and generates output values.

In the Number Guessing App, these output values represent the model's predictions for the possible digits.

4.6 Interpreting the Output Results

The model generates output values for different digit classes.

In a typical digit classification model, there are ten output classes representing digits from 0 to 9.

The application examines the output values and selects the digit with the highest prediction score.

For example, if the highest prediction score belongs to digit 2, the application predicts that the user has drawn the number 2.

Finally, the predicted digit is displayed on the application screen.

5 Overview and Purpose of the Provided GitHub Repository

The teacher provided an existing GitHub repository to help students understand the practical implementation of TensorFlow Lite in Android applications.

Repository Name: NumberGuessingAppTflite

GitHub Repository Link:

https://github.com/Uzair390-Del/NumberGuessingAppTflite

5.1 Purpose of the Repository

The purpose of this repository is to provide an existing Android application project that students can study, execute, and use as a reference for understanding TensorFlow Lite integration.

It helps students understand how an Android application is structured and how machine learning functionality can be incorporated into a mobile application.

By exploring the repository, students can learn about:

- Android project structure.
- Gradle configuration and dependencies.
- Application source code.
- TensorFlow Lite model integration.
- Input processing and prediction.
- Displaying results through an Android user interface.

5.2 Repository Cloning Procedure

Cloning means creating a local copy of an existing GitHub repository on a computer.

The repository can be cloned using the following steps:

1. Open GitHub and visit the provided repository link.
2. Click the green Code button.
3. Copy the HTTPS repository URL.
4. Open Android Studio or Git Bash.
5. Clone the repository to the local system.
6. Wait for the repository to download.
7. Open the downloaded project folder in Android Studio.
8. Allow Gradle Sync to complete.
9. Connect an Android device or start an emulator and execute the application.

5.3 Repository Structure

The provided GitHub repository contains the following main folders and files:

File/Folder	Purpose
`.idea`	Contains Android Studio project-related configuration.
`app`	Contains the main Android application module, including source code and resources.
`gradle`	Contains Gradle wrapper configuration files.
`.gitignore`	Specifies files and folders that Git should ignore.
`build.gradle`	Contains project-level Gradle configuration.
`gradle.properties`	Contains Gradle-related configuration properties.
`gradlew`	Gradle wrapper script for Unix-based systems.
`gradlew.bat`	Gradle wrapper script for Windows.
`settings.gradle`	Defines the Gradle project and its included modules.

The `app` folder is particularly important because it contains the Android application's implementation.

The application module generally includes source code, layouts, resources, and configuration files.

6 Analysis of the GitHub Repository

The provided repository is studied to understand how an existing Android application can be developed and how its components are organized.

The main purpose of this analysis is to understand the relationship between the Android interface and the machine learning model.

6.1 How the Model is Loaded

A TensorFlow Lite model must be loaded before the application can generate predictions.

In an Android application, the model file is commonly stored inside the assets folder.

For this project, the model used is `digit_model_TFLite`.

The Android application accesses the model file and loads it into memory. The TensorFlow Lite Interpreter is then initialized using the loaded model.

The model is loaded when required during application initialization, rather than being trained every time the application runs.

6.2 How Predictions are Generated

The application generates predictions through the TensorFlow Lite Interpreter.

The prediction process works as follows:

1. The user draws a digit on the application screen.
2. The application captures the drawing.

3. The drawing is converted into the required image format.
4. The input is processed according to the model's requirements.
5. The processed input is passed to the Interpreter.
6. The Interpreter executes the model.
7. The model generates output scores for the possible digits.
8. The application selects the digit with the highest output score.

This process allows the application to recognize handwritten digits using a trained machine learning model.

6.3 Input and Output Handling

6.3.1 Input Handling

The input is the handwritten digit drawn by the user.

The application captures the drawing and converts it into a format suitable for the model.

The input data may include:

- Image or bitmap data.
- Image dimensions.
- Pixel values.
- Grayscale values.
- Normalized numerical values.
- Input buffers.

The exact input dimensions, data type, and buffer shape depend on the model configuration.

6.3.2 Output Handling

The model produces numerical output values for the possible digit classes.

For a standard digit classifier, the output contains ten values representing digits from 0 to 9.

The application compares these values and selects the class with the highest score.

The selected digit is then converted into readable text and displayed to the user.

6.4 How Results are Displayed in the User Interface

The Android user interface provides a drawing area where users can enter handwritten digits.

After the prediction process is completed, the application displays the recognized digit on the screen.

The user interface acts as a communication layer between the user and the AI model.

The general interaction includes:

- Drawing a digit.
- Submitting the drawing for recognition.
- Processing the input through the model.
- Displaying the predicted digit.
- Allowing the user to try another digit.

This makes the application simple and interactive.

6.5 Importance of Code Comments

Code comments are important because they explain the purpose and functionality of different parts of the source code.

In this project, comments are used to explain:

- Model loading and initialization.
- Interpreter configuration.
- Input image processing.
- Buffer preparation.
- Model inference.
- Output processing.
- Displaying prediction results.

Proper comments make the application easier to understand, maintain, and modify.

7 Proposed AI-Based Application

7.1 Project Introduction

The proposed project is an AI-based Android application named Number Guessing App.

It is designed to recognize handwritten digits using a trained TensorFlow Lite model.

The application allows users to draw a number on the screen. The AI model analyzes the drawing and predicts which digit has been drawn.

The model used in this application is named `digit_model_TFLite`.

The application demonstrates how machine learning can be integrated into Android applications to perform image classification.

7.2 Purpose of the Application

The main purpose of the Number Guessing App is to demonstrate handwritten digit recognition through on-device artificial intelligence.

It provides a simple way to understand how an AI model receives image input, processes it, and generates a prediction.

The application also demonstrates the practical use of TensorFlow Lite in Android development.

7.3 Objectives of the Application

- To develop a simple AI-based Android application.
- To recognize handwritten digits from 0 to 9.
- To integrate the `digit_model_TFLite` model.
- To perform predictions using the TensorFlow Lite Interpreter.
- To process user drawings as model input.
- To display prediction results in the Android interface.
- To understand the practical use of on-device machine learning.

8 Technologies, Tools, Libraries, and AI Model Used

The following technologies and tools are used in the project:

Technology/Tool	**Purpose**
Android Studio	Development environment for creating the Android application.
Kotlin/Java	Android application programming language, according to the project's implementation.
XML	Used for designing Android user interface layouts where applicable.
TensorFlow Lite	Used to run the trained AI model on the Android device.
`digit_model_TFLite`	Model used for handwritten digit recognition.
Gradle	Manages project configuration and dependencies.
Git	Used to clone and manage the source code.
GitHub	Used to access the provided repository and manage project code.
Android Emulator/Physical Device	Used to run and test the application.

9 System Design and Workflow

9.1 System Design

The Number Guessing App consists of three main components:

1. **User Interface**
   The interface allows users to draw handwritten digits and view prediction results.

2. **Image Processing Component**
   This component captures and prepares the user's drawing according to the input requirements of the model.

3. **AI Model Component**
   The TensorFlow Lite model processes the prepared image and predicts the recognized digit.

9.2 Application Workflow

The application follows these steps:

1. The user opens the Number Guessing App.
2. The application initializes the required components and loads the AI model.
3. The user draws a digit on the drawing area.
4. The application captures the drawing.
5. The image is preprocessed according to the model's requirements.
6. The processed input is passed to the TensorFlow Lite Interpreter.
7. The model performs inference.
8. The application receives the output values.
9. The highest-scoring digit is selected.
10. The predicted digit is displayed on the screen.

System Workflow:

User Input → Draw Handwritten Digit → Image Processing → TensorFlow Lite Interpreter → AI Model Prediction → Output Processing → Display Predicted Digit

10 Main Features and Functionality

The Number Guessing App provides the following main features:

1. **Handwritten Digit Input**
   Users can draw digits directly on the application screen.

2. **AI-Based Digit Recognition**
   The application uses the `digit_model_TFLite` model to recognize handwritten digits.

3. **On-Device Prediction**
   The model performs inference directly on the Android device.

4. **Prediction Results**
   The application displays the digit predicted by the AI model.

5. **Interactive Interface**
   The user can interact with the drawing area and request digit recognition.

6. **Simple and User-Friendly Design**
   The application provides an easy way to enter digits and view the prediction.

11 Implementation Methodology

The implementation of the Number Guessing App follows a series of development steps.

Step 1: Repository Study
The provided GitHub repository is cloned and opened in Android Studio. Its folders, files, dependencies, and source code are explored.

Step 2: Project Setup
The Android project is configured and the required dependencies are synchronized using Gradle.

Step 3: Model Integration
The `digit_model_TFLite` model is included in the Android application in the appropriate location.

Step 4: Interpreter Initialization
The TensorFlow Lite Interpreter is initialized to load and execute the model.

Step 5: User Input Processing
The application captures the user's handwritten drawing and prepares it according to the model's input requirements.

Step 6: Model Inference
The prepared input is passed to the Interpreter, which executes the trained model and generates prediction values.

Step 7: Output Processing
The application identifies the output class with the highest prediction score.

Step 8: Result Display
The predicted digit is displayed through the Android user interface.

Step 9: Application Testing
The application is executed on an Android device or emulator to verify its functionality.

12 Testing, Results, and Discussion

Testing is an important part of application development because it helps verify whether the application works according to its requirements.

The Number Guessing App is tested by providing different handwritten digits as input and checking the predicted results.

12.1 Testing Procedure

1. Open the application on an Android device.

2. Draw a digit in the drawing area.
3. Submit the drawing for recognition.
4. Allow the application to process the input.
5. Observe the predicted digit displayed on the screen.
6. Repeat the process using different digits.

13 Conclusion

The Number Guessing App project provides practical experience in integrating Machine Learning and Deep Learning models into Android applications using TensorFlow Lite.

Through this project, the process of loading a trained model, preparing input data, running inference, and displaying prediction results is studied.

The project also provides an opportunity to explore an existing GitHub repository and understand its structure, configuration, and implementation.

The use of the `digit_model_TFLite` model demonstrates how handwritten digits can be recognized directly on an Android device without requiring continuous communication with an external server.

Overall, this project improves understanding of Android development, artificial intelligence, model integration, and on-device machine learning.